

Practice Set: 09

Section 1: Conceptual & Practical Questions (1–10)

Question 1: Define what a **Function** is in C++. Explain the modular philosophy behind using functions, highlighting the DRY principle, abstraction, debugging, and call stack memory efficiency.

Question 2: Explain the architectural mechanics of the **Call Stack** and **Stack Frame (Activation Record)** during a function call and return lifecycle.

Question 3: Differentiate among **Function Declaration (Prototype)**, **Function Definition**, and **Function Call**. What are the ordering rules regarding whether a function is defined above or below the `main()` function?

Question 4: Define and contrast **Actual Arguments** and **Formal Arguments** in terms of their definitions and locations in source code.

Question 5: Explain the **Call by Value** parameter passing mechanism. What memory allocations occur, and why do changes made to formal parameters not affect the caller's actual arguments?

Question 6: Explain the **Call by Address** parameter passing mechanism. Detail how pointer formal parameters and the address-of operator (`&`) allow modifications to the original variables.

Question 7: Explain the **Call by Reference** parameter passing mechanism. How does it achieve direct memory aliasing without allocating duplicate data copies or requiring explicit pointer dereferencing?

Question 8: Compare **Call by Value**, **Call by Address**, and **Call by Reference** across five key criteria: Formal Parameter Type, Call Site Syntax, Memory Created, Original Value Impact, and Syntax Cleanliness.

Question 9: What is a **Dangling Reference** bug? Explain why returning a reference to a local stack variable from a function causes severe runtime issues.

Question 10: Write three separate C++ functions named `addValue(int a, int b)`, `addAddress(int *a, int *b)`, and `addReference(int &a, int &b)` demonstrating the three parameter-passing mechanisms and invoking each inside `main()`.

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): Are parameter variable names mandatory in a C++ function prototype declaration?

Question 12 (Viva): What happens to a function's stack frame (activation record) in RAM when the function encounters a return statement?

Question 13 (Viva): What compilation error occurs if a function is defined *after* `main()` without a prototype declared above `main()`?

Question 14 (Viva): In which parameter passing method do formal parameters become direct memory aliases for actual arguments without allocating separate value storage?

Question 15 (Viva): Why is using `const` Type & considered a best practice for passing large structs, objects, or strings to functions?

Question 16 (Viva): If a function header is `void swap(int *x, int *y)`, how must the actual arguments be passed at the call site in `main()`?

Question 17 (Viva): Why does Call by Value consume more stack memory when passing large data structures compared to Call by Reference?

Question 18 (Viva): What is stored inside an activation record on the call stack when a function is invoked?

Question 19 (Viva): Can an actual argument passed by reference be modified without writing the indirection / dereference operator (*)?

Question 20 (Viva): Why should you avoid returning a reference (`int&`) to a local variable initialized inside a function block?

Answers & Explanations

Answer: 01

Definition: A Function is a named, self-contained block of statements designed to execute a specific computational task.

Modular Philosophy:

Code Reusability & DRY: "Don't Repeat Yourself" allows writing algorithmic logic once and invoking it repeatedly.

Abstraction & Modularity: Decomposes complex systems into smaller sub-problems for readability and organization.

Ease of Debugging & Testing: Isolates execution units for targeted testing and bug localization.

Memory Efficiency: Stack memory is allocated dynamically on the Call Stack during execution and automatically freed on completion.

Answer: 02

Call Stack Lifecycle:

A discrete memory block called a **Stack Frame (Activation Record)** is allocated on top of the call stack when a function is invoked.

The caller pushes pushed arguments/formal parameters, local variables, and the CPU Return Address into this stack frame.

Control jumps to the function's instructions; upon hitting return, the stack frame is popped (deallocated), restoring control to the caller.

Answer: 03

Declaration (Prototype): Informs compiler of the function's return type, name, and parameter types before invocation (return_type name(type1, type2);). Parameter names are optional.

Definition: Contains the actual executable body; parameter names are mandatory (return_type name(type1 p1) { ... }).

Call: Transfers CPU execution flow by supplying actual arguments (name(arg1);).

Ordering Rules: If defined *before* the call site, the prototype is optional. If defined *after* the call site (e.g., below main()), an explicit prototype declaration above main() is mandatory to prevent "undeclared identifier" errors.

Answer: 04

Actual Arguments: The real variables or literal values passed into the function at the call site (located inside the call statement, e.g., sum(a, b);).

Formal Arguments: The placeholder parameter variables defined in the function header to receive the passed data (located in the function declaration/definition header, e.g., int sum(int x, int y)).

Answer: 05

Mechanism: Formal parameters are declared as ordinary variables (e.g., int x).

Memory Allocation: Actual values are copied into separate, newly created stack slots for the formal parameters.

Original Value Impact: Modifications made inside the function only affect the local copies and have zero effect on the caller's actual arguments because they occupy distinct RAM addresses.

Answer: 06

Mechanism: Formal parameters are declared as pointer variables (e.g., `int *p`).

Memory Allocation: The call site passes the memory addresses of the actual arguments using `&` (e.g., `sum(&a, &b)`).

Modifications: Dereferencing the pointers using `*p` grants direct access to modify the physical RAM locations of the original variables in the calling function.

Answer: 07

Mechanism: Formal parameters are declared as reference variables (e.g., `int &x`).

Memory Allocation: Zero memory is allocated for duplicate copies or pointer holders; formal parameters become direct aliases for the actual variables.

Clean Syntax: Directly altering `x` modifies the caller's variable without requiring pointer syntax or explicit dereferencing (`*`).

Answer: 08

Feature	Call by Value	Call by Address	Call by Reference
Formal Parameter Type	Ordinary Variables (<code>int x</code>)	Pointer Variables (<code>int *p</code>)	Reference Variables (<code>int &x</code>)
Call Site Syntax	<code>sum(a, b);</code>	<code>sum(&a, &b);</code>	<code>sum(a, b);</code>
Memory Created	Duplicate copy created in stack.	Pointers allocated to hold addresses.	Zero copy allocation (Direct alias).
Original Value Impact	Original variables remain unmodified.	Original variables can be modified.	Original variables can be modified directly.

Feature	Call by Value	Call by Address	Call by Reference
Syntax Cleanliness	Simple.	Requires explicit * and & symbols.	Cleanest (No dereferencing needed).

Answer: 09

Dangling Reference: Occurs when a reference refers to a memory location that has been destroyed or deallocated.

Stack Deallocation Issue: When a function exits, its local variables allocated on the stack frame are immediately popped and invalidated. Returning a reference to a local stack variable leaves the caller with a reference pointing to invalid/corrupted RAM.

Answer: 10

```
#include <iostream>
using namespace std;
// 1. Call by Value
int addValue(int x, int y) {
    return (x + y);
}
// 2. Call by Address
int addAddress(int *p, int *q) {
    return (*p + *q);
}
// 3. Call by Reference
int addReference(int &x, int &y) {
    return (x + y);
}
int main()
{
    int a = 5, b = 6;

    int res1 = addValue(a, b);
    int res2 = addAddress(&a, &b);
    int res3 = addReference(a, b);

    cout << "Call by Value Result: " << res1 << endl;
    cout << "Call by Address Result: " << res2 << endl;
    cout << "Call by Reference Result: " << res3 << endl;

    return 0;
}
```

Answer: 11

No, parameter names in prototype declarations are optional dummy identifiers; only data types are required (e.g., `int sum(int, int);`).

Answer: 12

The stack frame is popped (deallocated) from the Call Stack, freeing its local variables and returning execution control to the return address.

Answer: 13

The compiler issues an "undeclared identifier" or "function not declared in this scope" compilation error.

Answer: 14

Call by Reference (`int &x`).

Answer: 15

It avoids expensive copy operations on large memory objects while ensuring read-only protection against accidental modifications.

Answer: 16

Memory addresses must be passed using the address-of operator (`&`), such as `swap(&a, &b);`.

Answer: 17

Call by Value must duplicate and store all object byte members into a new activation record on the stack, whereas Call by Reference passes only an alias without duplicating object memory.

Answer: 18

Formal parameter values/addresses, caller return address, and local variable allocations.

Answer: 19

Yes, references feature automatic dereferencing, allowing direct assignment syntax (e.g., `x = 20;`).

Answer: 20

Local variables inside a function are allocated on its stack frame and are destroyed upon returning, creating a dangerous dangling reference to invalid memory